

Ballot Builder — Three Flows Overview

Internal design document for sync meeting. Describes the three main flows, their current implementation status, data lifecycle, and open questions.

High-Level Summary

Ballot Builder is a ballot-recommendation assistant that (1) discovers a voter's policy preferences through natural conversation, (2) collects and structures candidate/ballot data along 15 civic policy axes plus 10 Schwartz motivational values, and (3) matches user values to candidates and ballot measures with transparent, per-axis reasoning.

The system is split into three flows because they operate on different cadences and have different compute/cost profiles:

- **Flow 1 (Values Capture)** runs once per user, is interactive and latency-sensitive — best served by a fast, cheap model with structured extraction prompts designed by a stronger model.
- **Flow 2 (Data Gathering)** runs once per election per geography, is offline and heavyweight — uses a strong model (Claude Opus) for scoring nuance, with cheaper models (Sonnet) for web research and output formatting.
- **Flow 3 (Matching)** runs on every page load, is pure math with no LLM involved — deterministic functions over structured data.

Two-model strategy (implemented): The conversational warmup (Flow 1) uses DeepInfra-hosted open-source models via explicit prompt harnesses designed with Claude. The research/scoring pipeline (Flow 2) uses Claude Sonnet for discovery and research, Claude Opus for scoring judgment. Flow 3 is entirely client-side TypeScript — no model needed.

Flow 1: User Onboarding / Values Capture

What exists today

Two parallel paths to the same profile format:

Path	Status	UX	Model
Swipe Assessment (original)	Production-ready	15 slider questions per domain, direct axis mapping	None (deterministic scoring)
Conversational Assessment (new)	Working prototype on demo/conversational-v2	Voice/text chat through 5 domains, then per-ballot-item discussion	DeepInfra OSS (two-pass)

Both paths produce a **BlueprintProfile** — the same data structure consumed by Flow 3.

Conversational flow: end-to-end

Phase 1: State Select

- User picks a state (currently MI, NC, GA, TX)
- Loads ballot items for that state
- Determines which domains are relevant to the ballot

Phase 2: Demographics (optional)

- Age, housing, insurance, income, employment
- Conditional: only asks fields relevant to ballot axes
- Stored in demographicStore for demographic-impact callouts later

Phase 3: Domain Warmup (the core values capture)

- For each relevant domain (econ → health → housing → justice → climate):
 1. System generates an opener question (LLM, temp 0.85)
 2. User responds via text or voice (Whisper transcription)
 3. Two-pass LLM processing (in parallel):
 - Pass 1 – Response: warm, conversational reply (temp 0.85)
 - Pass 2 – Extraction: structured ValueSignal[] (temp 0.3, JSON mode)
 4. If any signal has confidence < 0.4 → Template B refinement pass
 5. Signals merged into progressive profile via weighted averaging
 6. On domain complete → Template C summary for user review
 7. Advance to next domain

Phase 4: Ballot Items

- For each item (ordered by knowledge gaps first):
 1. Immediate recommendation computed from existing profile
 2. User can accept, skip, or "tell me more"
 3. Further discussion refines profile + updates recommendation
 4. User records vote or skips

Phase 5: Summary

- Vote summary, axes discovered, profile overview

Inputs, processing, outputs per turn

Step	Input	Processing	Output
User message	Free-text or voice transcript	Whisper STT → raw text	<code>string</code>
Pass 1 (Response)	Message + conversation history + domain context	LLM with neutrality/style constraints	Conversational reply <code>string</code>
Pass 2 (Extraction)	Message + axis definitions + current profile	LLM with scoring rubric + reference positions	<code>ValueSignal[]</code>
Template B (optional)	Low-confidence signals + user message	Classification LLM (multiple choice)	Refined signals
Signal validation	Raw signals from LLM	Server-side rules (clamp, quote-check, neutral-detection)	<code>sanitizedSignals[]</code>

Step	Input	Processing	Output
Profile merge	Validated signals + existing profile	Weighted average by confidence × signal count	Updated <code>ProgressiveAxisValue</code> record

Handling messy language

The prompt harness ([docs/PROMPT_HARNESS.md](#)) addresses these cases explicitly:

- **Ambiguity / hedging:** Confidence reduced (0.2–0.4), direction still extracted if discernible. "I guess maybe the government should help" → `econ_safetynet: 3.0, confidence: 0.3`.
- **Double negatives / self-corrections:** Extraction prompt instructs the model to resolve to final meaning. "I don't think we shouldn't have regulations" → positive regulation stance. If genuinely contradictory, produce conflicting signals with low confidence and flag `hasContradiction: true`.
- **"I don't know" / silence:** Detected as `userIntent: "skip"`. System offers a concrete example or moves on. Never fills with neutral 5.
- **Abstract values without policy mapping:** "I believe in fairness" → no axis signal extracted (not scorable). System asks a follow-up grounding it in policy: "When you think about fairness in housing, do you mean...?"
- **Vague agreement:** "Yeah, that makes sense" → no new signal extracted. System asks a different angle.

When to ask more vs. move on

- **Domain complete** when: the extraction LLM sets `domainComplete: true` OR the turn count reaches 3+ for that domain.
- **Ready for ballot** when: the last relevant domain is complete.
- **Skip available:** Users can skip any domain or the entire warmup. Skipped domains use defaults (neutral with low confidence — these contribute less to matching via confidence weighting).

Data persistence

```
// Stored in Zustand → localStorage (conversationStore)
ConversationSession {
  phase: 'state-select' | 'demographics' | 'warmup' | 'ballot'
  selectedState, selectedBallotId
  currentDomainIndex, domainTurnCount
  warmupMessages: ConversationMessage[]
  progressiveProfile: Record<axisId, ProgressiveAxisValue>
  itemConversations: Record<itemId, BallotItemConversation>
  completedCount, currentItemIndex
}

// ProgressiveAxisValue per axis
{
  value: number // 0-10 position
  confidence: number // 0-1 (how much evidence)
  importance: number // 0-10 (how much user cares)
  signalCount: number // number of contributing signals
}
```

Validation and feedback loops

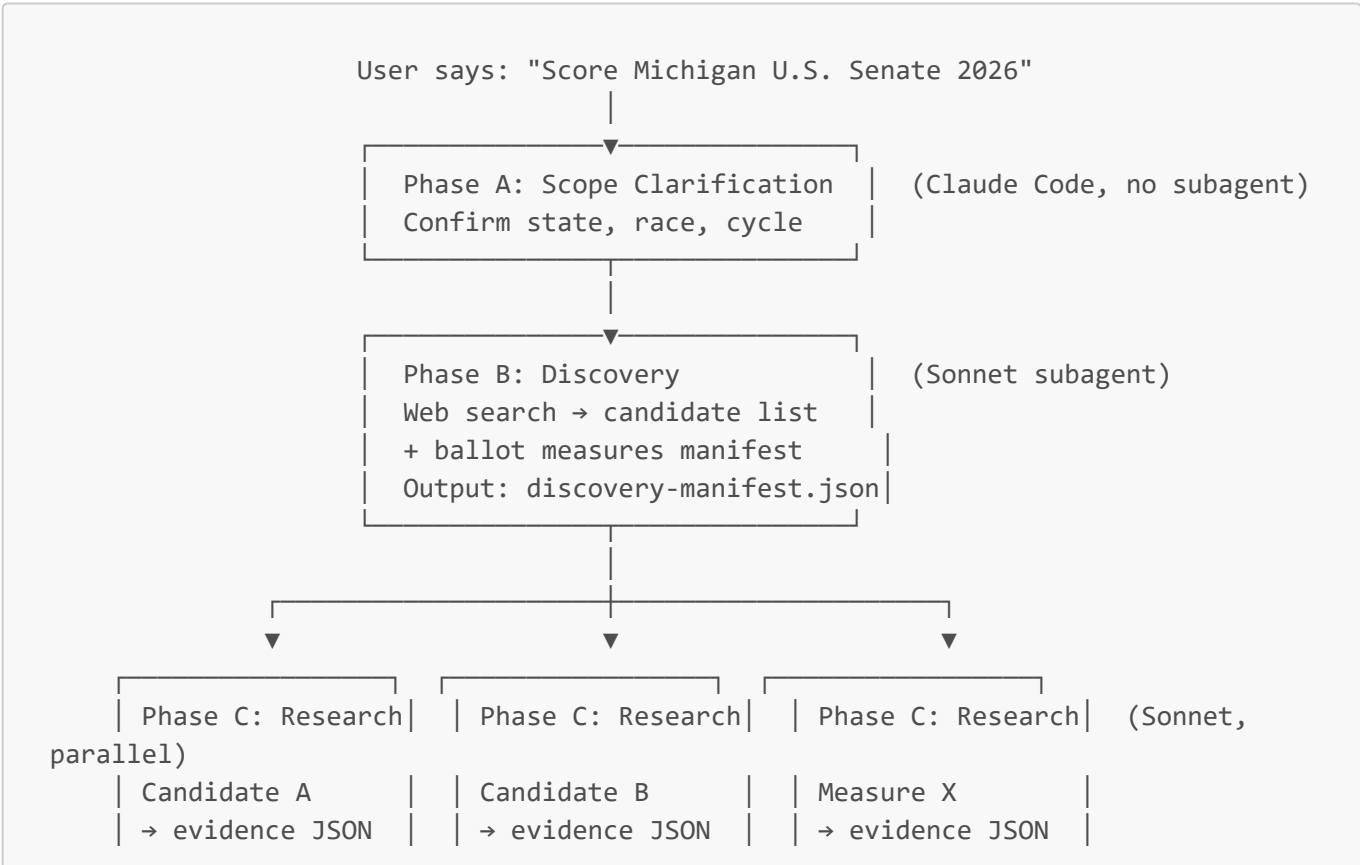
- **Server-side signal validation** (`signalValidation.ts`): rejects unknown axes, clamps out-of-range values, verifies source quotes exist in user message, demotes suspicious neutral scores (direction=5 with confidence >0.6).
- **Template C domain summaries**: after completing a domain, the system generates a plain-English summary ("It sounds like you lean toward X on economic policy...") that the user can implicitly confirm or correct.
- **User corrections**: "That's not what I meant" triggers re-extraction with the correction as context. The system doesn't overwrite — it adds a new signal with higher recency weight.

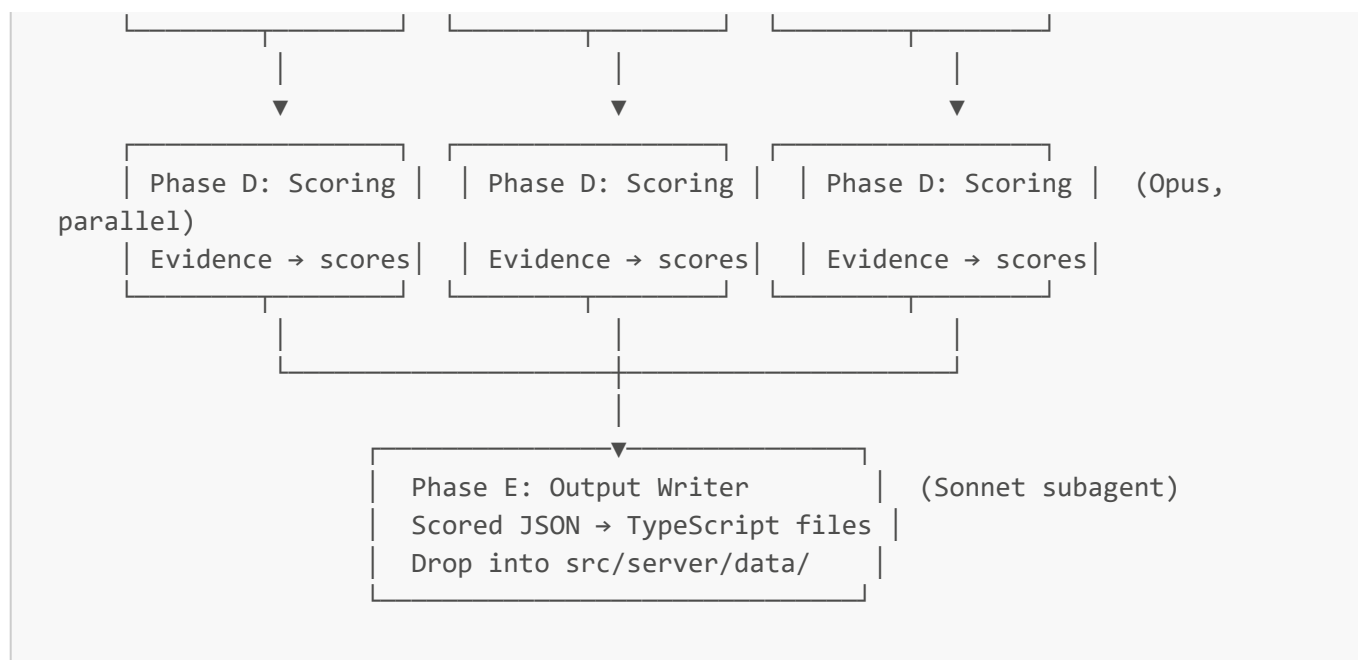
Flow 2: Data Gathering for Ballots / Candidates / Policies

What exists today

Component	Status	Location
Static ballot data (MI, NC, GA, TX)	Production-ready	<code>src/server/data/ballot/</code> — TypeScript files with manually curated scores
Agentic research pipeline	Working, used for all 4 states	<code>.claude/agents/</code> — 4 subagents orchestrated by Claude Code
Live API integration	Not started	Planned: Google Civic + Ballotpedia + VoteAmerica

Current pipeline (agentic, semi-automated)





Research layers (per candidate, in priority order)

1. **Campaign website** — issues/platform pages
2. **Voting record** — GovTrack, VoteSmart, state legislature databases
3. **Third-party scorecards** — LCV, NRA, Chamber of Commerce, AFL-CIO, Planned Parenthood, NARAL, Heritage Action, etc.
4. **Interviews** — state-specific outlets (e.g., WDET, Michigan Radio, MLive for MI)
5. **News coverage** — position-specific searches, endorsement coverage

Evidence hierarchy (from [SCORING_METHODODOLOGY.md](#))

Tier	Source	Trust Level
1	Voting records, official scorecards	Highest — objective, verifiable
2	Campaign websites, candidate interviews	High — direct from candidate
3	News articles, endorsement analysis	Medium — third-party interpretation
4	Background, biography, party affiliation	Lowest — inference only

When sources conflict: Tier 1 beats lower tiers. Voting record beats stated position. Recency favored.

Scoring output shape

```

// Per candidate (stored in scored/[slug].json, then compiled to candidates.ts)
{
  civicAxes: {
    econ_safetynet: { score: 3.0, confidence: "HIGH", sources: ["AFL-CIO 85%",
"Voted YES on HB 1234"] },
    climate_ambition: { score: 2.0, confidence: "MEDIUM", sources: ["LCV 91%"] },
    // ... only axes with evidence
  },
  schwartzValues: {
  
```

```

    universalism: { score: 0.7, confidence: "HIGH" },
    security: { score: -0.3, confidence: "MEDIUM" },
    // ...
  },
  profileSummary: "Progressive economic stance with strong environmental
record...",
  keyPositions: ["Supports $15 minimum wage", "Co-sponsored Green New Deal"],
  notes: ["2024 position on X conflicts with 2022 vote on Y"]
}

```

One-time vs. per-user work

Work Type	Frequency	Cost
Discovery (candidate list)	Once per election cycle per state	~\$1 agent cost
Research (evidence gathering)	Once per candidate	~\$2-3 per candidate
Scoring (evidence → axis scores)	Once per candidate	~\$3-5 per candidate (Opus)
Output compilation	Once per state	~\$1
Total per state	Once	~\$22 for a full state
Ballot lookup (which items for this address)	Per user	Free (static data) or API call
Matching (user profile × candidate scores)	Per page load	Free (client-side math)

Planned live API integration (not yet built)

```

User enters address
→ Google Civic API (free, 25K/day)
  → Districts, polling places, elected officials
→ Geocode to lat/long
→ Ballotpedia API (paid, single-state MI initially)
  → Full ballot: candidates, measures, elections
→ Cache in BallotCache table (Prisma)
  → Hash(district + election) → TTL 24h
→ Transform to internal types (match existing static data shape)
→ Serve to client

```

Geographic rollout strategy

- **Current:** 4 states with static data (MI, NC, GA, TX)
- **MVP target:** Michigan with live ballot lookup via Ballotpedia API

- **Expansion:** One state at a time. Each new state requires a pipeline run (~\$22, ~2 hours with human review) plus Ballotpedia API access expansion.
- **Scaling lever:** The output-writer agent is being updated to produce drop-in TypeScript, reducing manual integration from 1-3 hours to ~15 minutes per state ([docs/PIPELINE_SCALING_PLAN.md](#)).

Known bias and security considerations

Documented in [docs/PIPELINE_BIAS_AND_SECURITY.md](#):

- **Source selection bias:** Scorecard organizations don't cover all 15 axes evenly. Housing and policing have the fewest third-party scores.
- **Prompt injection risk:** Campaign websites could embed hidden instructions in HTML that survive Markdown conversion. No protections exist yet. Mitigations planned: [mcp-safe-fetch](#), content sanitization, adversarial review step.
- **Incumbent advantage:** Voting records and scorecards favor incumbents with legislative histories. First-time candidates have fewer scored axes — this is by design (we don't fabricate scores), but it means newer candidates may appear less "matched" simply due to less data.

Flow 3: Matching & Recommendation

What exists today

Fully implemented, deterministic, client-side. No LLM involved.

Two matching frameworks (both active)

A. Civic Axis Matching (primary)

For candidate races ([computeCandidateMatches\(\)](#)):

```
For each candidate:
  For each shared axis:
    diff = |userValue - candidateValue|    // both 0-10 scale
    weightedDiff += diff × axisWeight      // weight from user importance
    totalWeight += axisWeight

  avgDiff = weightedDiff / totalWeight
  matchPercent = max(0, (1 - avgDiff/10) × 100)

Alignment labels:
  diff ≤ 1 → "strong agreement"
  diff ≤ 3 → "moderate agreement"
  diff ≤ 5 → "weak agreement"
  diff > 5 → "opposed"
```

For ballot measures ([computePropositionRecommendation\(\)](#)):

```

For each relevant axis:
  yesEffect = measure.yesAxisEffects[axis] // -0.9 to +0.9 (polarity)
  userPref = (userValue - 5) / 5           // normalize to -1..+1
  alignment = yesEffect × userPref
  score += alignment × weight

normalizedScore = score / totalWeight
vote = "yes" if > 0.15, "no" if < -0.15, null otherwise
confidence = min(|normalizedScore| × 1.2, 1.0)

```

B. Schwartz Value Matching (secondary)

Same structure but operates on the 10 motivational values (universalism, benevolence, etc.) instead of policy axes. Uses candidate **valueStances** (-1 to +1) compared against user's ipsatized value scores.

C. Demographic Insights (supplementary)

Rule-based callouts: "As a renter, this zoning measure would..." Configured per ballot item with **demographicImpacts** mapping.

Handling missing/low-confidence axes

- **Missing axis on candidate:** That axis is excluded from the distance calculation — it doesn't count for or against.
- **Low-confidence user axis:** The conversational flow sets **importance** proportional to signal strength. Low-confidence axes get lower importance → lower weight in matching.
- **Confidence in recommendation:** For measures, confidence scales with alignment strength. A 0.16 normalized score produces a low-confidence "lean yes"; a 0.8 produces high-confidence.

Transparency and explainability

Every recommendation includes:

```

CandidateMatch {
  matchPercent: number           // headline number
  keyAgreements: string[]        // "You both support broader safety nets"
  keyDisagreements: string[]     // "You differ on gun policy"
  axisComparisons: [{
    axisName: "Safety Net Breadth"
    userValue: 3.2                // "Lean toward broader safety net"
    candidateValue: 7.8           // "Conditional support"
    alignment: "opposed"
    candidateEvidence: [{ text: "Voted against SNAP expansion", url: "..."}]
  }]
}

PropositionRecommendation {
  vote: "yes" | "no" | null
  confidence: number
}

```



```
    explanation: string          // "Based on your values, voting YES aligns
with..."
    factors: string[]           // Top 2-3 driving axes
    breakdown: [{
      axisName: "Climate Ambition"
      userStanceLabel: "Aggressive action"
      yesAlignsWith: "Aggressive action"    // ← agrees with user
      noAlignsWith: "Gradual approach"
      alignment: "yes"
    }]
  }
```

Guardrails

- Matching is deterministic — same inputs always produce same outputs, making it auditable.
- No single axis can dominate: weighted average ensures balance.
- Threshold for recommendation: measures need normalized score > 0.15 to get a YES/NO. Within the dead zone → "no clear recommendation."
- No LLM in the loop — eliminates hallucination risk in the final recommendation.

Where OSS models fit (conversational mode only)

In the conversational flow, the matching still uses the same deterministic functions. The LLM's role ends at signal extraction — it never produces the recommendation itself. The prompt harness explicitly instructs: "You are a translator, not a recommender."

Data Persistence & Validation Across Flows

Key data objects

Object	Created In	Storage	Key Fields	Validation
BlueprintProfile	Flow 1 (swipes or conversation)	Zustand → localStorage	Per-axis: value (0-10), confidence (0-1), importance, source, learning_mode	Shrinkage scoring (swipes), signal validation (conversation), user can lock/edit
SchwartzValueScores	Flow 1 (vignette assessment)	Zustand → localStorage	Per-value: raw_mean, ipsatized (-2..+2), n_answered	Ipsatization centers on individual mean
DemographicProfile	Flow 1 (optional gate)	Zustand → localStorage	age, housing, insurance, income, employment, state	Conditional fields based on ballot axes
CandidateScores	Flow 2 (pipeline)	Static TypeScript files	Per-axis: score (0-10), confidence level, evidence sources	Multi-tier evidence hierarchy, Opus scoring judgment

Object	Created In	Storage	Key Fields	Validation
BallotDefinition	Flow 2 (static or API)	TypeScript files or BallotCache table	contests, measures, candidates, election metadata	Cross-checked against Ballotpedia, SOS websites
MatchResult	Flow 3 (runtime)	Computed on render, not persisted	matchPercent, agreements, disagreements, axis breakdowns	Deterministic math — auditable by design
ConversationSession	Flow 1 (conversation mode)	Zustand → localStorage	phase, domain progress, messages, progressive profile, votes	Server-side signal validation per turn

Illustrative schemas

```
// User's progressive axis profile (conversation mode)
type ProgressiveProfile = Record<string, {
  value: number // 0-10, current best estimate
  confidence: number // 0-1, evidence strength
  importance: number // 0-10, how much user cares
  signalCount: number // contributing observations
}>

// Candidate axis stance (from pipeline)
type CandidateAxisStance = Record<string, number> // axisId → 0-10 score
// Plus evidence:
type AxisEvidence = Record<string, SourceRef[]> // axisId → [{text, url}]

// Ballot item (union type)
type BallotItem = Contest | Measure
// Contest has candidates[] with axisStances, valueStances
// Measure has yesAxisEffects, yesValueEffects, relevantAxes
```

Feedback loops

- User corrections in conversation:** "That's not what I meant" → system adds a corrective signal, doesn't overwrite. Higher recency weight shifts the profile.
- Domain summaries:** After each domain warmup, Template C generates a summary. User's reaction (explicit correction or implicit acceptance) feeds back.
- Slider overrides:** In the swipe assessment path, users can manually adjust any axis slider. This sets `source: "user_edited"` and `learning_mode: "dampened"` — future swipes blend 80/20 with the user's edit.
- Locked axes:** Users can freeze an axis entirely (`locked: true, learning_mode: "frozen"`). No further automated updates.

5. **Pipeline corrections:** If a scored candidate file has errors, the output-writer can be re-run after manual evidence correction. The pipeline is idempotent.

Re-validation over time

- **User profiles:** No expiration. Users can retake assessments or edit sliders at any time.
 - **Candidate scores:** Effectively static per election cycle. Could be re-scored if new evidence emerges (e.g., a vote is cast mid-campaign).
 - **Ballot cache** (planned): 24-hour TTL on API-fetched ballot data. Stale entries re-fetched.
 - **Demographic data:** No expiration, but users can update. Some fields are election-contextual (e.g., state selection).
-

Open Questions / Design Decisions to Discuss

1. Axis count: 15 or 16?

The prompt mentions 16 axes including a "tax policy" axis. The current spec has 15 (5 domains × 3). Do we add a tax policy axis? If so, which domain does it belong to (Economic?) and what are its poles? This affects the assessment item count, candidate scoring, and all matching math.

2. Quantitative scores vs. qualitative tags — how much of each?

The system currently uses numeric scores (0-10) with qualitative labels at the poles. Should we lean more into qualitative tagging (e.g., "strong environmental advocate" rather than "climate_ambition: 2.3")? The conversational flow already extracts both, but the matching engine only uses numbers.

3. How aggressive should follow-ups be?

Currently: 3 turns per domain max, then move on. Template B fires for low-confidence signals. Should we be more persistent (5 turns if confidence is still low) or more respectful of user time (2 turns, accept gaps)?

4. Local races with sparse data

Down-ballot races (school board, county commissioner) often have zero scorecard ratings, no voting record, and minimal campaign websites. Options:

- Skip scoring, show only basic info (name, party, background)
- Use LLM analysis of whatever is available (campaign Facebook posts?) with explicit LOW confidence
- Let users add their own notes/ratings for these races

5. Conversational vs. swipe assessment — eventual convergence?

Both produce a [BlueprintProfile](#). Should we:

- Keep both paths permanently (user choice)?
- Phase out swipes in favor of conversation?
- Use swipes as a quick fallback when voice/text isn't practical?

6. Profile portability across elections

A user's civic values don't change much between elections, but ballot items do. Should we carry forward profiles and just re-run matching against new ballots? Or prompt users to re-assess (at least partially) each cycle?

7. Confidence thresholds for showing recommendations

Currently: measures need normalized score > 0.15 for a YES/NO. Candidates need match > 50% for "best match." Are these thresholds right? Too aggressive = false confidence. Too conservative = too many "no recommendation" results.

8. Real-time vs. batch candidate scoring

Current pipeline is batch (run once, produce static files). Live API integration would enable real-time scoring of newly filed candidates. But real-time scoring means LLM calls on the hot path — cost and latency concerns. Hybrid approach: batch for known candidates, flag unknowns for next pipeline run?

9. Multi-model cost optimization for conversation

The warmup currently makes 2-4 LLM calls per user turn (response + extraction + optional refinement + optional summary). At scale (100K users × 15 turns average), that's 3-6M LLM calls per election cycle. Should we:

- Reduce passes (single-pass extraction with inline response)?
- Cache common openers/summaries?
- Use an even cheaper model for Pass 1 (response only)?

10. Handling partisan perception

Even with neutral axes and transparent evidence, users may perceive the system as biased based on which candidates score highest for them. Mitigations:

- Show full axis breakdowns always (not just summary match %)
- Let users adjust importance weights post-hoc and see how it changes results
- Show the scoring methodology in-app ([docs/ABOUT_BALLOT_BUILDER.md](#) exists but isn't prominently linked)
- Consider a "bias audit" report showing score distributions across parties

Appendix: Current Implementation Status

Component	Status	Branch
Civic Blueprint Assessment (swipes)	Complete	main
Schwartz Values Assessment (vignettes + boosters)	Complete	main
Archetype Classification	Complete	main
Static Ballot Data (MI, NC, GA, TX)	Complete	main
Candidate/Measure Matching Engine	Complete	main

Component	Status	Branch
Demographic Insights	Complete	main
Conversational Values Capture	Working prototype	demo/conversational-v2
Voice Input (Whisper STT)	Working prototype	demo/conversational-v2
Signal Extraction Prompt Harness	Working prototype	demo/conversational-v2
Agentic Research/Scoring Pipeline	Working, used for all 4 states	main (agents in .claude/agents/)
Live Ballot API Integration	Not started	—
Authentication (Clerk)	Not started	—
Database Schema Expansion	Not started	—
Match Evidence UI Component	Not started	—